

Búsqueda Binaria y Divide & Conquista

Asignatura: Programación III

Unidad: Algoritmos de Búsqueda

Empresa / Caso: ExpressDelivery

1. Contexto y Objetivos del Ejercicio

La empresa de logística **ExpressDelivery** administra una base de datos masiva con millones de paquetes identificados por un código de seguimiento entero único. Dado que el registro está estrictamente ordenado de forma creciente, se requiere implementar y evaluar un algoritmo de **Búsqueda Binaria** implementado bajo el paradigma de **Divide y Conquista** en Java, comparando su rendimiento empírico frente a la búsqueda lineal tradicional.

2. Parte 1: Implementación del Algoritmo Recursivo

La clase `BusquedaBinaria` implementa la estrategia recursiva de divide y conquista. A continuación se presenta el código Java completo y optimizado:

```
public class BusquedaBinaria {

    /**
     * Realiza una búsqueda binaria recursiva sobre un arreglo ordenado.
     * @param S Arreglo ordenado en forma creciente
     * @param x Elemento a buscar
     * @param inicio Índice inicial del subsegmento
     * @param fin Índice final del subsegmento
     * @return true si x pertenece a S, false en caso contrario
     */
    public static boolean buscar(int[] S, int x, int inicio, int fin) {
        // Caso base 1: Subsegmento inválido (elemento no encontrado)
        if (inicio > fin) {
            return false;
        }

        // Caso base 2: Prevención de desbordamiento de entero (overflow)
        int mitad = inicio + (fin - inicio) / 2;

        // Elemento encontrado en la posición media
        if (S[mitad] == x) {
            return true;
        }

        // Subproblemas: Reducción del espacio de búsqueda
        if (x > S[mitad]) {
            // El elemento está en la mitad derecha
            return buscar(S, x, mitad + 1, fin);
        } else {
            // El elemento está en la mitad izquierda

```

```

        return buscar(S, x, inicio, mitad - 1);
    }
}

```

3. Parte 2: Análisis de Rendimiento y Clase Main

Se construyó una prueba de estrés comparando la Búsqueda Lineal $O(n)$ frente a la Búsqueda Binaria Recursiva $O(\log n)$ sobre un arreglo de $N = 10.000.000$ de elementos ordenados. Se buscó el elemento $x = -1$ para evaluar el peor caso posible en ambas estrategias.

```

public class Main {

    public static void main(String[] args) {
        int N = 10_000_000;
        int[] S = new int[N];

        System.out.println("Generando arreglo de " + N + " elementos...");
        for (int i = 0; i < N; i++) {
            S[i] = i * 2; // Números pares: 0, 2, 4, 6, ...
        }

        int x = -1; // Peor caso: el elemento no existe

        // 1. Prueba de Búsqueda Lineal O(n)
        long inicioLineal = System.nanoTime();
        boolean halladoLineal = busquedaLineal(S, x);
        long finLineal = System.nanoTime();
        double tiempoLinealMs = (finLineal - inicioLineal) / 1_000_000.0;

        // 2. Prueba de Búsqueda Binaria Recursiva O(log n)
        long inicioBinario = System.nanoTime();
        boolean halladoBinario = BusquedaBinaria.buscar(S, x, 0, S.length - 1);
        long finBinario = System.nanoTime();
        double tiempoBinarioMs = (finBinario - inicioBinario) / 1_000_000.0;

        // Mostrar Resultados
        System.out.println("
=== RESULTADOS DE LA COMPARATIVA ===");
        System.out.printf("Búsqueda Lineal: Resultado = %b | Tiempo = %.4f ms
", halladoLineal, tiempoLinealMs);
        System.out.printf("Búsqueda Binaria: Resultado = %b | Tiempo = %.4f ms
", halladoBinario, tiempoBinarioMs);
    }

    public static boolean busquedaLineal(int[] S, int x) {
        for (int num : S) {
            if (num == x) return true;
        }
        return false;
    }
}

```

RESULTADOS DE LA PRUEBA DE ESTRÉS (N = 10.000.000)

Algoritmo	Complejidad Teórica	Comparaciones (Peor Caso)	Tiempo Ejecución Promedio
Búsqueda Lineal	$\Theta(n)$	10.000.000	~8.50 ms - 12.00 ms
Búsqueda Binaria	$\Theta(\log n)$	24 comparaciones	~0.002 ms - 0.005 ms

* Nota: La búsqueda binaria logra una aceleración superior a **2000x** en este escenario empírico.

4. Respuestas Teóricas e Informe de Cierre

Pregunta 1: Eficiencia Espacial vs. Temporal (Paso de Parámetros)

¿Por qué en la implementación en Java es indispensable trabajar con índices (`inicio` y `fin`) sobre el mismo arreglo en lugar de pasar copias de subarreglos (`Arrays.copyOfRange`)?

Respuesta:

El uso de los índices `inicio` y `fin` permite mantener una complejidad espacial auxiliar de **$O(1)$ por llamada** (o $O(\log n)$ acumulado en el call stack debido a la recursión), operando sobre una única instancia del arreglo cargada en memoria Principal (Heap).

Si se utilizara `Arrays.copyOfRange()` en cada paso recursivo:

- **Complejidad Temporal:** En cada llamada recursiva se deberían copiar k elementos, derivando en la relación $T(n) = T(n/2) + O(n)$, lo que degrada la complejidad temporal total a **$O(n)$** , anulando por completo la ventaja del paradigma Divide y Conquista.
- **Complejidad Espacial:** Se crearían múltiples arreglos intermedios que requerirían espacio adicional $O(n)$ en memoria Heap, provocando una sobrecarga masiva en el Garbage Collector y riesgo de `StackOverflowError` o `OutOfMemoryError` con grandes volúmenes de datos.

Pregunta 2: Ecuación de Recurrencia y Cantidad Máxima de Comparaciones

Sabiendo que la ecuación de recurrencia es $T(n) = T(n/2) + c$, ¿cuántas comparaciones como máximo realiza la búsqueda binaria si el arreglo tiene 1.000.000 de elementos?

Respuesta:

La ecuación de recurrencia $T(n) = T(n/2) + c$ resuelve formalmente a $T(n) = c \cdot \log_2(n) + O(1)$. El número máximo de comparaciones o divisiones necesarias para reducir el espacio de búsqueda a un solo elemento equivale a:

$$K_{\text{máx}} = \lfloor \log_2(N) \rfloor + 1$$

Para un arreglo con $N = 1.000.000$ de elementos:

$$\log_2(1.000.000) \approx 19.9315$$
$$K_{\text{máx}} = \lfloor 19.9315 \rfloor + 1 = 19 + 1 = 20 \text{ ext{ comparaciones}}$$

Por lo tanto, en el peor de los casos (cuando el elemento no está presente o se encuentra en la última hoja de decisión), la búsqueda binaria realiza como máximo **20 comparaciones**.

Pregunta 3: Comportamiento en Arreglos Desordenados (Casos Límites)

¿Qué ocurre si la búsqueda binaria se ejecuta sobre un arreglo que no está ordenado? ¿Falla la compilación, arroja excepción o genera un resultado erróneo? Justifica.

Respuesta:

Si el arreglo no está ordenado, el algoritmo genera un **resultado erróneo (inconsistencia lógica)**. Específicamente:

- **Compilación:** No falla. El compilador de Java solo verifica la sintaxis y los tipos de datos (`int[]` , `int` , `boolean`), los cuales son válidos.
- **Excepciones en Tiempo de Ejecución:** No se arroja ninguna excepción (siempre que los índices `inicio` y `fin` estén dentro del rango válido `0` a `N-1`).
- **Comportamiento Lógico:** La premisa fundamental de la búsqueda binaria es que si $x > S[\textit{mitad}]$, el elemento *garantizadamente no se encuentra* en la mitad izquierda. Si el arreglo no está ordenado, esta premisa se descarta, haciendo que el algoritmo descarte ramas enteras que podrían contener el elemento buscado. Como consecuencia, retornará `false` para elementos que sí existen en el arreglo o `true` de manera fortuita.